End-to-End Assignment

Complete MLOps Pipeline Implementation

Objective

Design and implement a complete MLOps pipeline that includes:

- Data versioning using DVC + S3
- Automated pipeline using GitHub Actions or Jenkins
- Experiment tracking using MLflow
- Model deployment using FastAPI + Docker
- Container publishing using Docker Hub
- Inference validation

Mandatory Identification Requirement

> ⚠ **Missing identification in any of the components below will result in penalty.**

Students must include their Name and Roll Number in ALL the following places:

Component	Requirement
GitHub Repository Name	<rollno>-mlops
Docker Image Name	<dockerhub-username>/<rollno>-mlops
MLflow Experiment Name	<rollno>_experiment
API /health or / endpoint	Must return Name + Roll No
/predict response	Must include Name + Roll No
Output metrics JSON	Must include Name + Roll No

Part 1: Problem Definition

Students must clearly describe:

1. Problem Statement

- Type: Classification / Regression / Other
- Objective of the model

2. Dataset Description

- Source of dataset
- Features
- Target variable
- Dataset size
- Preprocessing steps

Part 2: Repository Setup

Students must create a GitHub repository:

> **<rollno>-mlops-assignment**

The repository must include:

- Training script

- API code
- Dockerfile
- CI/CD workflow
- DVC configuration

Part 3: Data Versioning (DVC + S3)

Students must:

1. Initialize DVC
2. Configure S3 as remote storage
3. Create at least two dataset versions:
   - Version 1 → partial / modified dataset
   - Version 2 → full / improved dataset
4. Push both versions to S3

Part 4: CI/CD Pipeline

Students must use **ONE** of the following: GitHub Actions *or* Jenkins.

Pipeline must include:

1. Code checkout
2. Dependency installation
3. DVC data pull
4. Model training
5. MLflow logging
6. Metrics generation

Part 5: MLflow Experiment Tracking (Mandatory – 5 Runs)

Students must:

1. Use experiment name: **<rollno>_experiment**
2. Perform **at least 5 runs**, varying one or more of:

- Dataset version (v1 vs v2)
- Model type
- Hyperparameters
- Feature selection (subset of features used for training)

Example Variation Strategy

Run	Dataset	Model	Change Applied
Run 1	Version 1	Model A	Base configuration (all features)
Run 2	Version 1	Model A	Hyperparameter change
Run 3	Version 2	Model A	Base configuration
Run 4	Version 2	Model A	Feature selection (reduced features)
Run 5	Version 2	Model B	Different model + feature selection

Feature Selection Requirement (Mandatory)

At least one run must include feature selection, where:

- A subset of features is used
- Selected features must be documented
- Impact on performance must be analyzed

Each run must log:

- Parameters
- Metrics (at least 2)
- Model artifact

Part 6: Model Deployment (FastAPI)

Endpoint 1: Health Check

> **/health OR /**

Response must include:

- Name: <student_name>
- Roll No: <rollno>

Endpoint 2: Prediction

> **/predict**

Response must include:

- Prediction
- Name
- Roll Number

Part 7: Dockerization

Students must:

1. Build Docker image
2. Tag image: **<dockerhub-username>/<rollno>-mlops**
3. Push image to Docker Hub

Part 8: Inference Validation

Students must:

1. Pull Docker image
2. Run container
3. Send test request
4. Verify response contains:
   - Prediction
   - Name
   - Roll No

Part 9: Results Comparison Across Runs

Students must create a comparison table and clearly compare all 5 runs:

Run	Dataset	Model	Key Parameters	Metric 1	Metric 2
Run 1					
Run 2					
Run 3					
Run 4					
Run 5					

Part 10: Reproducibility

Students must:

1. Checkout earlier commit
2. Run dvc pull
3. Reproduce one run
4. Verify metrics match

Analysis Questions

A. Run-Based Analysis

1. Which run performed the best? Why?
2. How did dataset changes affect performance?
3. How did hyperparameter tuning affect results?
4. How did feature selection impact performance?
5. Which run performed worst? Explain why.
6. Which had greater impact: data change or parameter change?

B. Experiment Tracking

1. How did MLflow help compare runs?
2. What information was most useful in selecting the best model?

C. Data Versioning

1. What differences were observed between dataset versions?
2. Why is data versioning critical in ML systems?

D. System Design

1. How does your pipeline ensure reproducibility?
2. What are the limitations of your pipeline?
3. How would you improve this system for production use?

Final Deliverable (Single Submission)

Students must submit **ONE comprehensive report (PDF)**.

Report Must Include

1. Student Details

- Name
- Roll Number

2. Problem Description

- Problem statement
- Dataset description

3. Implementation Steps

Detailed explanation of:

- DVC + S3 setup
- CI/CD pipeline
- MLflow integration
- API implementation
- Dockerization

4. Experiment Results

- Table of 5 runs
- Comparison
- Observations

5. Screenshots (Roll No/Name Must Be Visible)

Include:

1. GitHub repository
2. DVC tracking and S3 bucket
3. CI/CD pipeline execution
4. MLflow runs (all 5)
5. Docker image in Docker Hub
6. Running container
7. API response with Name + Roll No

6. Links

- GitHub repository link
- Docker Hub link

7. Answers to Analysis Questions

All answers must be included in the report.